Queue Data Structure

1. Introduction to Queue

A queue is a linear data structure in which insertion takes place at one end, called the REAR, and deletion takes place at the other end, called the FRONT.

A queue follows the FIFO (First In, First Out) principle. The element inserted first is removed first.

Real-life examples:

- People waiting in a ticket counter queue.
- Print jobs waiting for a printer.
- Processes waiting for CPU scheduling.
- Requests waiting on a web server.
- Messages waiting to be processed.

Visual representation of a queue

10

20

30

40

FRONT → Deletion

Insertion ← REAR

If 10 was inserted first, it will be removed first.

2. Basic Queue Operations

Operation	Description
enqueue(x)	Inserts element x at the rear.
dequeue()	Removes and returns the front element.
isEmpty()	Checks whether the queue is empty.
isFull()	Checks whether a fixed-capacity queue is full.
display()	Displays queue elements.

Queue overflow: Inserting into a full, fixed-capacity queue.

Queue underflow: Removing an element from an empty queue.

3. Types of Queues

The four important queue types:

1. Linear Queue

Insertion takes place at the rear and deletion at the front. In a simple array implementation, freed positions at the beginning may remain unused.

2. Circular Queue

The last array position connects logically to the first position. Freed spaces can be reused through wraparound.

3. Priority Queue

Each element has an associated priority. Elements are removed according to priority rather than simply by arrival order.

4. Double-Ended Queue (Deque)

Insertion and deletion are permitted at both the front and rear.

4. Linear Queue

A linear queue is the basic implementation of a queue using an array or list.

Working

Consider an initially empty queue of capacity 5.

Operation	Queue contents	FRONT	REAR
Enqueue 10	10	0	0
Enqueue 20	10, 20	0	1
Enqueue 30	10, 20, 30	0	2
Dequeue	20, 30	1	2
Enqueue 40	20, 30, 40	1	3

In a simple linear array queue, REAR advances toward the end of the array. Even when earlier positions become free, they cannot be reused without shifting elements or resetting the queue.

This limitation is called wastage of space or false overflow when free positions exist but the rear has reached the last array index.

5. Circular Queue

A circular queue is a queue in which the last position is logically connected to the first position. It overcomes the space-wastage problem of a simple linear array queue.

Working example

Consider a circular queue of capacity 5.

Circular queue: wraparound

60

Index 0

70

Index 1

30

Index 2

40

Index 3

50

Index 4

FRONT = 2

REAR = 1

Illustration after wraparound: elements 60 and 70 occupy positions 0 and 1, following 30, 40 and 50 in circular order.

Suppose the queue initially contains [10, 20, 30, 40, 50]. After two dequeue operations, positions 0 and 1 become available. New elements can be inserted at these positions by wrapping the rear around the array.

Conditions for a circular queue

For a circular queue of capacity \(size\):

- Empty: front == -1
- Full: (rear + 1) % size == front

Advantages:

- Efficient reuse of array positions.
- No shifting of elements is required.
- Enqueue and dequeue take \(O(1)\) time.

Applications: CPU scheduling, circular buffers, streaming data, keyboard buffers and network packet handling.

6. Priority Queue

A priority queue is a data structure in which every element has an associated priority. Elements are removed according to priority rather than just arrival time.

For this example, assume a smaller priority number represents a higher priority.

Example

Element	**Priority**	**Removal order**

Emergency patient	1	1st
Serious patient	2	2nd
General patient	3	3rd

If two elements have the same priority, they may be processed in arrival order if the implementation explicitly supports FIFO tie-breaking.

7. Double-Ended Queue (Deque)

A deque, pronounced “deck,” is a queue in which insertion and deletion are allowed at both ends.

Unlike a standard queue, which inserts at the rear and removes from the front, a deque supports operations at the front and rear.

Operations

Operation	**Description**
append(x)	Insert at rear
appendleft(x)	Insert at front
pop()	Remove from rear
popleft()	Remove from front
isEmpty()	Check whether empty

Types of deque

1. Input-restricted deque

- Insertion is allowed at only one end.
- Deletion is allowed at both ends.

2. Output-restricted deque

- Insertion is allowed at both ends.
- Deletion is allowed at only one end.

Applications:

- Sliding-window maximum and minimum algorithms.
- Palindrome checking.
- Work-stealing schedulers.
- Undo/redo history with bounded storage.
- Algorithms requiring insertion and removal from both ends.

8. Comparison of All Four Queue Types

Feature	**Linear Queue**	**Circular Queue**	**Priority Queue**	**Deque**
Processing order	FIFO	FIFO	Based on priority	Depends on operations
Insertion	Rear	Rear, with wraparound	According to implementation	Front and rear
Deletion	Front	Front, with wraparound	Highest-priority element	Front and rear
Reuses freed array positions	Not always	Yes	Depends on implementation	Depends on implementation
Typical implementation	Array/list	Circular array	Heap	Doubly linked list or circular array
Major application	Print queue	Circular buffer	CPU scheduling	Sliding-window algorithms